

Laborator 3

DPAC

As. drd. Dan Curăvale

- **KDD** vine de la *Knowledge Discovery and Data Mining*.
- **KDD Cup** este competitia anuala organizata de ACM SIGKDD pe probleme de data mining.
- In editia din **1999**, una dintre probe a fost *computer network intrusion detection*.
- Scopul era sa se construiasca un clasificator care separa conexiunile *normale* de cele *ilegale / de atac*.
- **DARPA 1998** a furnizat traficul brut (capturi de pachete + loguri), iar **KDD Cup 99** este varianta prelucrata in inregistrari de conexiune (41 features + label).

De ce a aparut NSL-KDD?

- KDD Cup 99 a devenit benchmark-ul clasic pentru IDS-uri, dar in timp i s-au observat limite serioase.
- Setul continea multe inregistrari duplicate sau foarte asemanatoare.
- Asta favoriza modelele care memorau exemple frecvente si producea scoruri bune in mod artificial.
- In **2009**, Mahbod Tavallae si colaboratorii au propus **NSL-KDD** ca revizie a setului original.
- Ideea principala: un subset mai curat si mai echilibrat, potrivit pentru comparatii mai corecte intre modele.

NSL-KDD: ce contine efectiv?

- O **instanta** reprezinta o **conexiune/flow** intre doua endpoint-uri (sursa → destinatie), agregata pe durata sesiunii.
- Structura este: **41 features + label + difficulty**, iar toate aceste campuri sunt **per conexiune**, nu **per tip de atac**.
- **Label** = clasa conexiunii: fie `normal`, fie numele unui atac (ex. `neptune`, `satan`).
- **Difficulty** = scor introdus in NSL-KDD pentru cat de greu e de clasificat instanta; nu este severitatea atacului.
- Din exemplele de mai jos: `duration`, `src_bytes` sunt **numerice**, iar `protocol_type`, `service`, `flag` sunt **categorice**.
- In setul complet apar **39 tipuri de atac**, grupate in 4 familii (DoS, Probe, R2L, U2R).
- Pentru modelare: binar (*normal vs attack*) sau multiclasa (*tip specific de atac*).

NSL-KDD: clase de atac (grupe)

Clasa	Idee principala	Exemple
Normal	trafic legitim, fara comportament malitios	normal
DoS	blocheaza sau degradeaza disponibilitatea serviciului	neptune, smurf
Probe	recunoastere, scanare, colectare de informatie	satan, nmap
R2L	atacator extern incearca acces fara cont valid	guess_passwd, warezclient
U2R	utilizatorul ajuns pe sistem incearca privilegii ridicate	buffer_overflow, rootkit

De ce il facem?

- intelegem forma reala a dataset-ului
 - verificam daca datele au sens inainte de modelare
 - identificam rapid probleme de calitate
-
- EDA raspunde la intrebarea: *ce trebuie curatat si pregatit inainte sa antrenam modelul?*

Ce urmarim?

- numar de randuri, coloane, tipuri de date
- distributia claselor
- distributii asimetrice (mp vs pret) si valori extreme
- coloane constante sau aproape constante (risc seismic)
- corelatii suspecte si zgomot (exemplu EEG)

Problema	Impact	Tratament tipic
Valori lipsa	unele modele nu accepta direct campuri incomplete	eliminare controlata sau imputare cu medie / mediana / mod
Outliers	pot domina distantele si pot distorsiona scaler-ul	inspectie, clipping, transformare log, scalare robusta
Dezechilibru de clase	modelul favorizeaza clasa majoritara si ignora cazurile rare	stratify, ponderi pe clase, resampling, metrice precum recall/F1

- In detectia intruziunilor, exact clasele rare pot fi cele mai importante pentru sistem.

Encoding: de ce avem nevoie de el?

- **Encoding** = procesul prin care transformam date categoriale (text) in valori numerice.
- Modelele ML opereaza cu calcule matematice, nu cu etichete text precum tcp, http sau SF.
- In NSL-KDD, coloane precum `protocol_type`, `service` si `flag` trebuie convertite inainte de antrenare.
- Alegerea metodei de encoding influenteaza direct ce relatie va invata modelul intre categorii.
- Intrebarea corecta nu este doar *cum codificam?*, ci si *ce presupunere introducem prin codificare?*

Label Encoding vs One-Hot Encoding

Metoda	Exemplu	Cand o folosim	Riscul principal
Label Encoding	mic, mediu, mare → 0, 1, 2	target sau categorii cu ordine naturala	introduce o ordine numerica
One-Hot Encoding	tcp, udp, icmp → [1, 0, 0], [0, 1, 0], [0, 0, 1]	categorii nominale, fara ordine	creste dimensionalitatea

- Pentru `protocol_type`, Label Encoding poate sugera fals ca `udp > tcp`; One-Hot evita aceasta ordine artificiala.
- Regula practica: **ordine reala** → Label Encoding; **fara ordine** → One-Hot.

Normalizare (Min-Max)

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- aduce valorile intr-un interval fix, de regula $[0, 1]$
- utila cand vrem scale comparabile si limite clare
- sensibila la outliers
- Daca vrei interval fix, alegi normalizare; daca vrei comparatie fata de medie si dispersie, alegi standardizare.

Standardizare (z-score)

$$z = \frac{x - \mu}{\sigma}$$

- centreaza datele in jurul mediei 0
- aduce abaterea standard aproape de 1
- utila pentru KNN, SVM, regresie logistica (acolo unde se lucreaza direct cu distante -> normalizarea poate face squash la marea majoritate a valorilor)

Train/Validation/Test Split

- Separarea in **train**, **validation** si **test** separa clar invatarea, validarea (alegerea modelului si a hiperparametrilor) si evaluarea finala.
- O impartire practica este 70/20/10: train pentru antrenare, validation pentru tuning, test pentru verificarea finala.
- Pentru clase dezechilibrate folosim split **stratificat**, astfel incat proportiile claselor sa ramana apropiate in toate subseturile.
- Fara aceasta separare, putem raporta scoruri optimiste deoarece deciziile de modelare vad indirect datele de evaluare.

Atentie la leakage

- `fit()` pentru scaler sau encoder se face doar pe train.
- Pe validation si test aplicam doar `transform()`.
- Daca informatia din validation/test intra in preprocessing, evaluarea nu mai este corecta.